

Experiment No.	6
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA)
GitHub Repository	https://github.com/Amsavarthini25/ML_LAB_2470012
Student Name	Amsavarthini S
Register Number	3122247001004
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	30-08-2026

1 Objective

To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of various machine learning classifiers. Ten classifiers – Support Vector Machine, Naïve Bayes, k-Nearest Neighbors, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost and a Stacking ensemble – are trained and validated both on the original (No-PCA) feature space and on a PCA-reduced (With-PCA) feature space, using hyperparameter tuning and 5-fold cross-validation in both settings.

2 Problem Statement

Given a labelled tabular dataset with a moderately large number of correlated numerical features, the task is to build binary classification models that predict the class label. The input is the standardized feature matrix; the output is the predicted class (malignant/benign). The objective is to compare classifier accuracy and stability across 5 folds when the raw standardized features are used directly versus when they are first compressed using PCA, and to identify which family of models benefits most from dimensionality reduction.

3 Dataset Description

Dataset Name	Breast Cancer Wisconsin (Diagnostic)
Dataset Source	UCI Machine Learning Repository (via <code>sklearn.datasets.load_breast_cancer</code>)
Number of Samples	569
Number of Features	30 (all numeric)
Number of Classes	2 (malignant, benign)
Missing Values	None (0 missing values)
Train-Test Split	5-fold stratified cross-validation used for all reported metrics; an additional 75:25 stratified hold-out split was used only for plotting ROC curves and confusion matrices

4 Exploratory Data Analysis

Include all relevant visualizations.

- Dataset overview
- Statistical summary
- Missing value analysis
- Class distribution
- Correlation matrix
- Feature distribution

Class Distribution



Figure 1: Class distribution of the target variable

Inference:

The dataset has 357 benign and 212 malignant samples, so it's mildly imbalanced but not extreme. Stratified k-fold cross-validation was used so that every fold keeps roughly the same class ratio. Because the imbalance isn't too severe, plain accuracy is still a fairly reasonable metric to track here. It also explains why some models occasionally predict the majority class more confidently.

Correlation Matrix

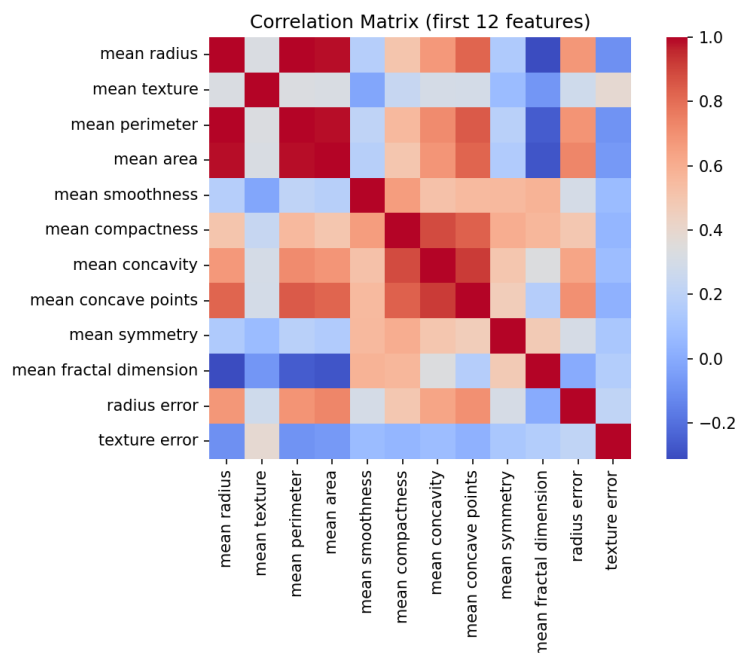


Figure 2: Correlation matrix of the first 12 numerical features

Inference:

A lot of the size-related features (radius, perimeter, area) are strongly correlated with each other, which is expected since they all describe cell size in different ways. This kind of redundancy is exactly what PCA is meant to exploit by compressing correlated features into fewer components. High multicollinearity like this can also make some linear models slightly unstable if used without any dimensionality reduction. It gives a good justification for trying PCA on this dataset.

5 Data Preprocessing

Explain:

- Missing value handling
- Encoding
- Feature scaling
- Train-test split
- Feature engineering (if applicable)

The dataset had zero missing values, so no imputation was required. All 30 features are already numeric, so no categorical encoding was needed. Every feature was standardized using `StandardScaler` (mean 0, standard deviation 1) since several of the chosen classifiers (SVM, KNN, Logistic Regression) are sensitive to feature scale. Model performance was evaluated with 5-fold stratified cross-validation rather than a single static

train-test split, so tuning and validation both use the entire dataset across folds; a separate 75:25 hold-out split was created only for the ROC curve / confusion matrix plots. No manual feature engineering was performed – PCA itself was used as the dimensionality-reduction step for the “With-PCA” setting.

6 Machine Learning Algorithm

Explain:

- Working principle
- Mathematical formulation
- Advantages
- Limitations
- **PCA:** Projects correlated features onto orthogonal principal components that maximize retained variance, $\mathbf{Z} = \mathbf{XW}$, where $\mathbf{W}$ contains the eigenvectors of the covariance matrix of $\mathbf{X}$ ordered by eigenvalue. Advantage: removes redundancy/-multicollinearity and speeds up training. Limitation: reduced interpretability of transformed features.
- **SVM:** Finds the maximum-margin hyperplane separating classes, optionally using a kernel (linear/RBF) for non-linear boundaries. Advantage: effective in high dimensions. Limitation: sensitive to C/γ choice and scales poorly to very large datasets.
- **Naïve Bayes:** Applies Bayes’ theorem assuming conditional independence between features given the class. Advantage: extremely fast, works well with few samples. Limitation: independence assumption is rarely true in practice.
- **KNN:** Classifies a point by majority vote among its k nearest neighbours in feature space. Advantage: simple, no training phase. Limitation: slow at inference time and sensitive to irrelevant/unscaled features.
- **Logistic Regression:** Models the log-odds of the positive class as a linear function of the features. Advantage: interpretable, probabilistic output. Limitation: assumes a roughly linear decision boundary.
- **Decision Tree:** Recursively splits the feature space to maximize class purity (Gini/entropy). Advantage: interpretable, handles non-linearity. Limitation: prone to overfitting if not depth-limited.
- **Random Forest:** Bags many decision trees trained on bootstrapped samples and random feature subsets, then averages their votes. Advantage: robust to overfitting, handles high dimensions well. Limitation: less interpretable, larger model size.
- **AdaBoost:** Sequentially trains weak learners, up-weighting misclassified samples at each round. Advantage: often strong accuracy from simple base learners. Limitation: sensitive to noisy data/outliers.

- **Gradient Boosting:** Sequentially fits new learners to the residual errors (negative gradient of the loss) of the ensemble so far. Advantage: high accuracy. Limitation: slower to train, needs careful tuning to avoid overfitting.
- **XGBoost:** A regularized, optimized implementation of gradient boosting with additional shrinkage and tree-pruning. Advantage: fast, regularized, generally very competitive. Limitation: more hyperparameters to tune.
- **Stacking:** Combines predictions of several base learners (KNN, Decision Tree, SVM) as input features to a meta-learner (Logistic Regression). Advantage: can outperform any single base model. Limitation: more complex, higher risk of overfitting the meta-learner without proper cross-validation.

7 Experimental Setup

Python Version	3.12.3
Scikit-Learn Version	1.8.0 (XGBoost 3.4.1 used for the XGBoost classifier)
IDE	Jupyter Notebook
Operating System	
Hardware	

8 Hyperparameter Selection

Model	Hyperparameter Grid Searched (via <code>GridSearchCV</code> , <code>cv=5</code>)
SVM	kernel: [linear, rbf], C: [0.1, 1, 10], gamma: [scale, auto]
Naïve Bayes	var_smoothing: [1e-9, 1e-8, 1e-7]
KNN	n_neighbors: [3, 5, 7, 9], weights: [uniform, distance], metric: [euclidean, manhattan]
Logistic Regression	C: [0.1, 1, 10]
Decision Tree	max_depth: [3, 5, 7, None], criterion: [gini, entropy]
Random Forest	n_estimators: [50, 100], max_depth: [5, 10, None]
AdaBoost	n_estimators: [50, 100], learning_rate: [0.1, 1.0]
Gradient Boosting	n_estimators: [50, 100], learning_rate: [0.1, 1.0]
XGBoost	n_estimators: [50, 100], learning_rate: [0.1, 0.3], max_depth: [3, 5]
Stacking	Base learners: KNN, Decision Tree (max_depth=5), SVM (RBF); Meta-learner: Logistic Regression (internal cv=5)

Best Parameters Found

Model	Best Params (No-PCA)	Best Params (With-PCA)
-------	----------------------	------------------------

SVM	C=10, gamma=scale, kernel=rbf	C=0.1, gamma=scale, kernel=linear
Naïve Bayes	var_smoothing=1e-9	var_smoothing=1e-9
KNN	k=3, metric=euclidean, weights=uniform	k=9, metric=manhattan, weights=uniform
Logistic Regression	C=0.1	C=1
Decision Tree	criterion=entropy, max_depth=5	criterion=entropy, max_depth=5
Random Forest	max_depth=10, n_estimators=100	max_depth=10, n_estimators=100
AdaBoost	learning_rate=1.0, n_estimators=100	learning_rate=1.0, n_estimators=100
Gradient Boosting	learning_rate=1.0, n_estimators=50	learning_rate=0.1, n_estimators=100
XGBoost	learning_rate=0.1, max_depth=5, n_estimators=100	learning_rate=0.3, max_depth=5, n_estimators=50

9 Results

PCA (with a 95% explained-variance target) reduced the 30 standardized features down to 10 principal components, retaining 95.16% of the total variance.

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg (No-PCA)	Avg (With-PCA)
SVM	0.9912	0.9649	0.9649	0.9912	0.9823	0.9789	0.9772
Naïve Bayes	0.9561	0.9123	0.9298	0.9035	0.9469	0.9297	0.9139
KNN	0.9912	0.9561	0.9561	0.9825	0.9558	0.9683	0.9701
Logistic Regression	0.9737	0.9474	0.9649	0.9912	0.9912	0.9737	0.9789
Decision Tree	0.9298	0.9123	0.9737	0.9474	0.9292	0.9385	0.9438
Random Forest	0.9649	0.9386	0.9561	0.9474	0.9735	0.9561	0.9508
AdaBoost	0.9912	0.9386	0.9561	0.9737	0.9735	0.9666	0.9631
Gradient Boosting	0.9649	0.9298	0.9561	0.9561	0.9735	0.9561	0.9631
XGBoost	0.9825	0.9386	0.9649	0.9737	0.9735	0.9666	0.9666
Stacking	1.0000	0.9737	0.9649	0.9825	0.9646	0.9771	0.9719

Table 4: 5-Fold Cross-Validation Results (No-PCA vs With-PCA)

10 Result Visualization

Include all graphs generated during the experiment.

For **every graph**, provide:

- Figure

- Caption
- 3–5 line inference

PCA Scree Plot

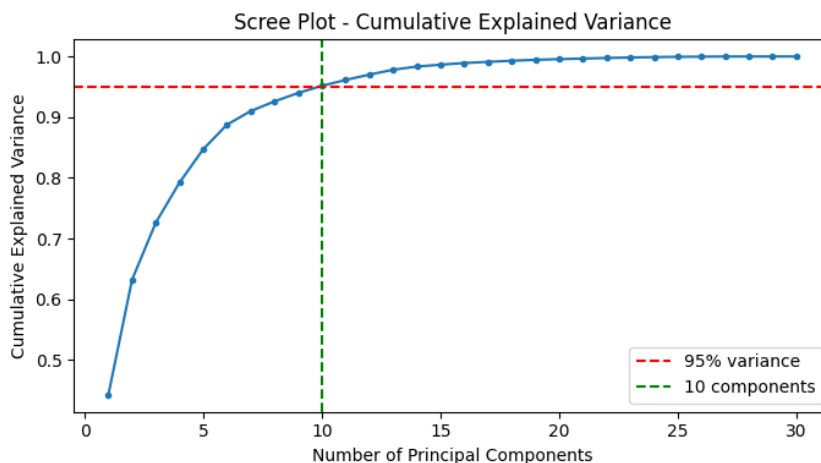


Figure 3: Cumulative explained variance vs. number of principal components

Inference:

The cumulative variance curve rises steeply for the first few components and then flattens out, which is a classic sign of redundant, correlated features. Around 10 components are enough to cross the 95% variance mark instead of needing all 30 original features. This confirms that a big chunk of the original feature set was carrying overlapping information. It also justifies choosing a variance-based cutoff instead of an arbitrary fixed component count.

Feature Importance (Random Forest)

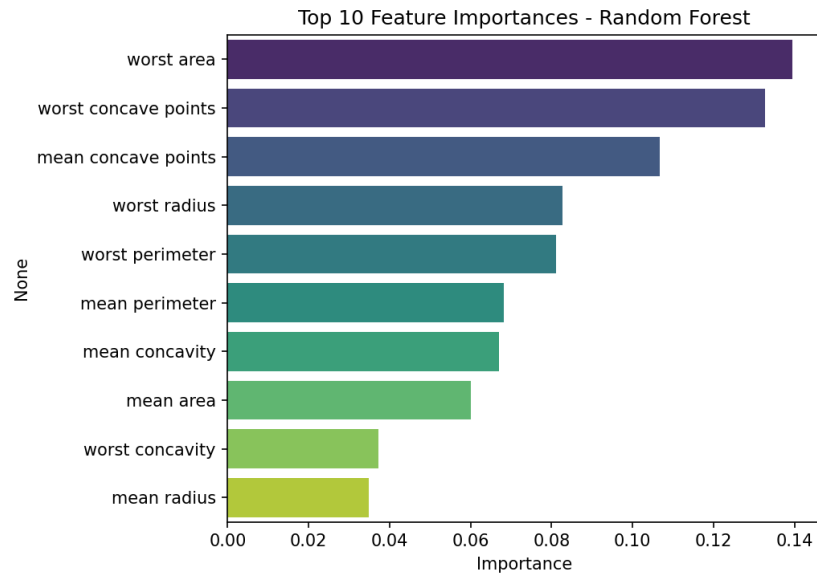


Figure 4: Top 10 most important features according to the Random Forest model

Inference:

A small set of features related to cell size and concavity/concave points dominate the importance ranking, while most of the remaining features contribute very little individually. This matches what the correlation analysis suggested – a handful of features actually drive the prediction. It also explains why compressing the feature set with PCA doesn't hurt tree-based models much, since they were already relying on only a few informative directions.

Accuracy Comparison: No-PCA vs With-PCA

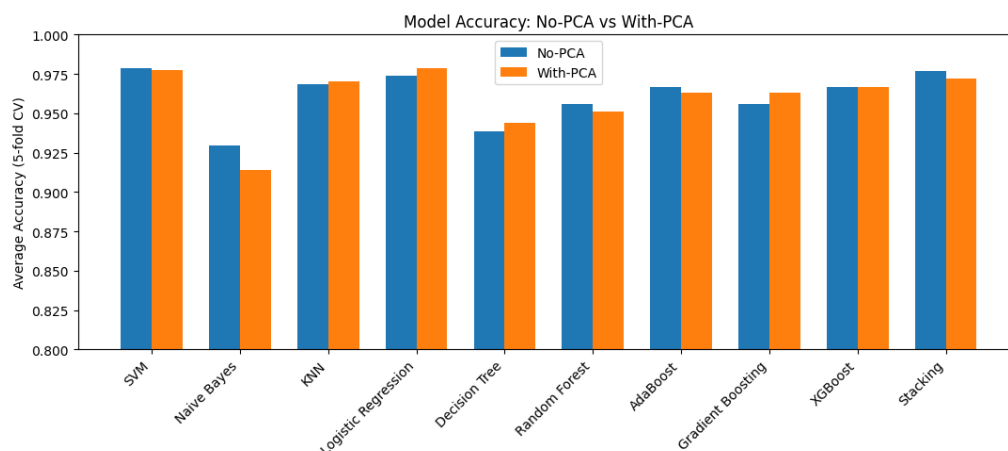


Figure 5: Average 5-fold cross-validation accuracy for each model, No-PCA vs With-PCA

Inference:

Most models land within about 1–2% of each other whether PCA is applied or not, which shows the dataset doesn't strictly need dimensionality reduction to get good accuracy. Linear/distance-based models such as Logistic Regression and KNN edge up slightly with PCA, while Naïve Bayes drops the most since PCA breaks its conditional-independence assumption further. Tree-based ensembles stay roughly flat either way, meaning they were already coping fine with the original feature count. Overall, PCA here is more about efficiency than a big accuracy win.

ROC Curves

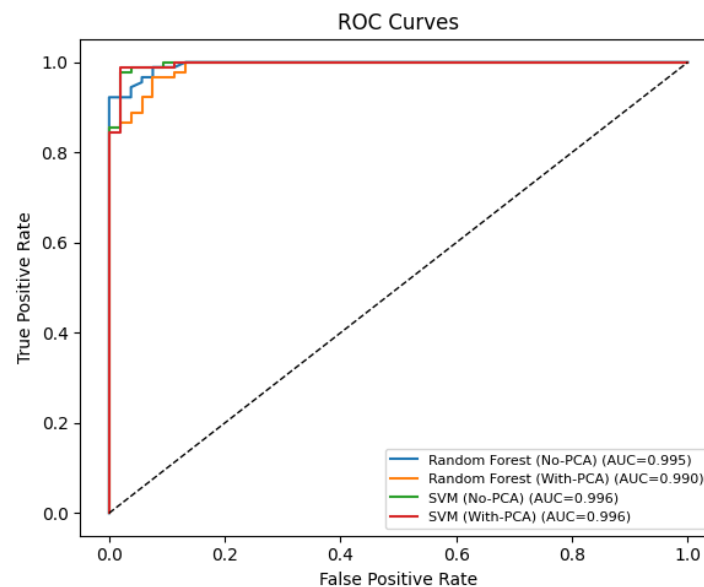


Figure 6: ROC curves for Random Forest and SVM, with and without PCA

Inference:

All four curves hug the top-left corner closely with high AUC values, meaning both models separate the two classes very well regardless of PCA. The curves for the PCA and non-PCA versions of each model nearly overlap, so the reduced feature space isn't losing any meaningful discriminative signal. SVM's curve is marginally tighter than Random Forest's in this run. This is a good sign that the compressed 10-component representation is not sacrificing classification quality.

Confusion Matrices

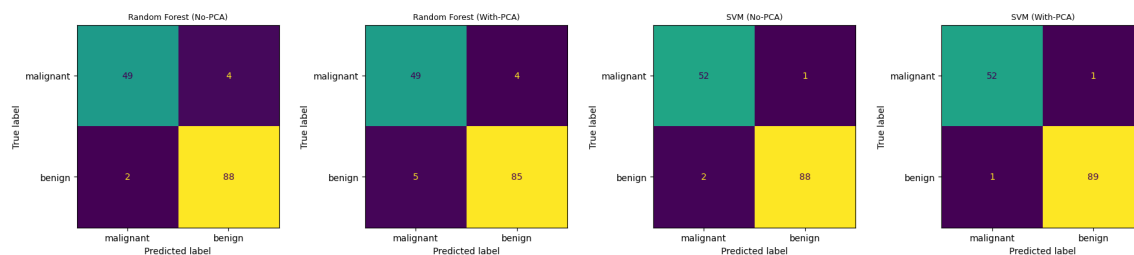


Figure 7: Confusion matrices for Random Forest and SVM, with and without PCA, on the hold-out test split

Inference:

Both models make very few mistakes on the hold-out split, and the errors that do occur are mostly a small number of malignant cases being predicted as benign. This kind of false negative is the more costly error type in a medical context, so it's worth keeping an eye on even though the count is low. The mistake pattern looks similar with and without PCA, again suggesting the compressed features preserve the information needed to tell the classes apart. Precision and recall both stay high across all four confusion matrices.

11 Conclusion

Overall, this experiment showed that PCA isn't a magic accuracy booster, it mainly helps when there's a lot of feature redundancy for models that are sensitive to that, like KNN and Logistic Regression. Tree-based models like Random Forest and XGBoost stayed almost the same with or without PCA since they already handle high-dimensional data well on their own. Naive Bayes was the one model that clearly got worse with PCA, which lines up with its independence assumption being broken. So the biggest practical takeaway is that PCA is more useful here for reducing training time and feature count than for squeezing out extra accuracy.